

University Student-Support Case Agent - Week 4 Tool Catalogue and Tool/API Schemas

Group A

Name	Student Number
Namatovu Christine Maria	2300701098
Opiyo Oscar	2300701330
Isio Lily	2300700445
Babirye Sandra	2300707270
Ogwal Richard	2300716574

BSE4104 Emerging Trends in Software Engineering Prepared against: Project Charter (Week 1 AI Boundary Matrix), Week 3 Corpus and RAG Design (DOC-01–DOC-12, TF-IDF retrieval), Week 3 RAG Testing & Grounding Failure Report (R08, R11, R13, R15), Prompt v3.0-RAG
Baseline model: Gemini 2.0 Flash - native function-calling /structured-output support

Design principle carried from Week 1–3: the orchestrator, not the model, is the authority on what a tool call is allowed to do. Every tool below enforces its authorization and validation checks in deterministic code before a result ever reaches the model, and the model can only choose whether and with what arguments to call a tool that already exists - it cannot create new capabilities. Section 4 lists the tools that were deliberately not built, because the Week 3 grounding-failure report (R13, R14) showed that boundary detection by phrasing alone is unreliable; the more robust fix is to make sure no tool exists that could carry out a prohibited action even if a boundary check were bypassed.

1. Tool Catalogue Summary

Tool	Purpose	Reads / Writes	Classification	Auth required
get_timetable	Return the authenticated student's class schedule for a semester	Read	Retrieves current application data	Session-bound student ID
get_case_status	Return the status and details of an existing support case	Read	Retrieves current application data	Session-bound student ID + case ownership
search_knowledge_base	Retrieve top-k relevant excerpts from the approved DOC-01–DOC-12 corpus	Read	Retrieves current application data (reference corpus)	None beyond an authenticated session
create_support_case	Draft a new support case from a student's description	Write	Performs a low-risk simulated side effect	Session-bound student ID

Two tools are read-only lookups against live application state (**get_timetable**, **get_case_status**); one is a read-only lookup against the static reference corpus (**search_knowledge_base**); one performs the catalogue's only side effect, and that side effect is deliberately low-risk: it appends a draft record with a status the model cannot change, has no external system integration, and requires human review before it has any real-world consequence (**create_support_case**).

2. Tool Definitions

2.1 get_timetable

Purpose: Look up the authenticated student's class timetable for a given semester from the deterministic timetable store, so the agent never has to generate a schedule from memory or from retrieved text.

Input schema

```
{
  "name": "get_timetable",
  "description": "Retrieve the authenticated student's timetable for a given semester.",
  "parameters": {
    "type": "object",
    "properties": {
      "student_id": {
        "type": "string",
        "pattern": "^[0-9]{10}$",
        "description": "Must equal the authenticated session's student ID."
      },
      "semester": {
        "type": "string",
        "pattern": "^[0-9]{4}-S[12]$",
        "description": "Defaults to the current semester if omitted."
      }
    },
    "required": ["student_id"],
    "additionalProperties": false
  }
}
```

Output schema

```
{
  "type": "object",
  "properties": {
    "student_id": { "type": "string" },
    "semester": { "type": "string" },
    "entries": {
      "type": "array",
      "items": {
```

```

    "type": "object",
    "properties": {
      "course_code": { "type": "string" },
      "day": { "type": "string" },
      "time": { "type": "string" },
      "room": { "type": "string" }
    },
    "required": ["course_code", "day", "time", "room"]
  },
  "retrieved_at": { "type": "string", "format": "date-time" }
},
"required": ["student_id", "semester", "entries", "retrieved_at"]
}

```

Authorization requirements: student_id in the request must match the authenticated session's student ID exactly; the tool never accepts a caller-supplied identity it cannot verify against the session. There is no “lookup on behalf of another student” mode.

Failure behavior: - Unknown student_id (not in data/students.json) → structured {"error": "not_found"}; the agent surfaces this as “I can’t find a timetable for that student ID,” never a fabricated schedule. - student_id present but does not match the session → hard refusal at the orchestrator layer before the tool executes; logged as a boundary event (same identity-boundary class of failure the RAG evaluation set’s Q15 was designed to catch). - Valid student with no entries for the semester → returns entries: [], not an omitted field, so the agent states “no classes on record” rather than staying silent or guessing.

2.2 get_case_status

Purpose: Look up the current status and metadata of a support case the student previously submitted.

Input schema

```

{
  "name": "get_case_status",
  "description": "Retrieve the status of a support case belonging to the authenticated student.",
  "parameters": {
    "type": "object",
    "properties": {
      "case_id": {
        "type": "string",
        "pattern": "^CASE-[0-9]{4}$"
      },
      "student_id": {
        "type": "string",
        "pattern": "^[0-9]{10}$",
        "description": "Must equal the authenticated session's student ID."
      }
    }
  }
}

```

```

    },
    "required": ["case_id", "student_id"],
    "additionalProperties": false
  }
}

```

Output schema

```

{
  "type": "object",
  "properties": {
    "case_id": { "type": "string" },
    "status": { "type": "string", "enum": ["Pending", "In Progress", "Resolved", "Rejected"] },
    "category": { "type": "string", "enum": ["missing_marks", "retake_request",
      "registration_issue", "timetable_clash"] },
    "subject": { "type": "string" },
    "created_at": { "type": "string", "format": "date-time" },
    "updated_at": { "type": "string", "format": "date-time" }
  },
  "required": ["case_id", "status", "category", "subject", "created_at", "updated_at"]
}

```

Authorization requirements: the case record’s stored `student_id` must match the session’s authenticated student ID. A student may only query cases they own. This is the deterministic control that directly prevents the “check on my roommate’s case” scenario the Week 3 RAG evaluation set flagged as an adversarial identity test.

Failure behavior: - `case_id` not found → `{"error": "not_found"}`. - `case_id` found but owned by a different student → refuse before returning any part of the record (not even the category or subject), logged as a boundary event; the agent’s response is identical in wording to a “not found” case, so the response itself does not leak whether the case ID exists for someone else. - Malformed `case_id` (fails the pattern) → validation error returned to the orchestrator, tool is not invoked, agent asks the student to confirm their case ID.

2.3 search_knowledge_base

Purpose: Retrieve the top-k most relevant excerpts from the approved DOC-01–DOC-12 corpus via the existing TF-IDF retriever, so every generated answer is built only from cited excerpts (Prompt v3.0-RAG’s “answer ONLY from excerpts” rule).

Input schema

```

{
  "name": "search_knowledge_base",
  "description": "Retrieve relevant excerpts from the approved knowledge corpus for grounded Q&A.",
  "parameters": {
    "type": "object",
    "properties": {

```

```

    "query": { "type": "string", "minLength": 3, "maxLength": 300 },
    "top_k": { "type": "integer", "minimum": 1, "maximum": 5, "default": 3 },
    "min_score": { "type": "number", "minimum": 0, "maximum": 1, "default": 0.14 }
  },
  "required": ["query"], "additionalProperties": false
}

```

Output schema

```

{
  "type": "object",
  "properties": {
    "query": { "type": "string" },
    "results": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "doc_id": { "type": "string", "pattern": "^DOC-[0-9]{2}$" },
          "score": { "type": "number" },
          "excerpt": { "type": "string", "maxLength": 220 },
          "source_file": { "type": "string" }
        },
        "required": ["doc_id", "score", "excerpt", "source_file"]
      }
    },
    "retrieved_at": { "type": "string", "format": "date-time" }
  },
  "required": ["query", "results", "retrieved_at"]
}

```

Authorization requirements: none beyond having an authenticated session — the corpus contains no student-specific data, so any logged-in student may query it. No case- or student-ownership check applies here.

Failure behavior: - No chunk scores above `min_score` (as in R08 and R15) → `results: []`. This is the signal the orchestrator uses to force an abstention (“I don’t know from approved documents”) rather than letting the model answer from parametric memory. - A result above `min_score` that is topically adjacent but does not actually answer the question (the R15 false-grounding failure) is a known limitation of lexical TF-IDF matching, not a tool-contract failure — the tool is returning exactly what it is specified to return (the best-scoring chunks), which is why Prompt v3.0-RAG’s answerability check, not the tool schema, is the intended second line of defense; the Week 3 report’s suggested fix (stronger relevance/answerability checks before grounding) is a prompt- and validator-level improvement, not a change to this schema. - Malformed query (empty, or exceeds `max-length`) → validation error, tool not invoked.

2.4 create_support_case

Purpose: Draft a new support case on the student's behalf once the required fields have been collected and confirmed, so a human in the department can review and act on it.

Input schema

```
{
  "name": "create_support_case",
  "description": "Create a draft support case for the authenticated student. Status is always server-
assigned and cannot be set by the caller.",
  "parameters": {
    "type": "object",
    "properties": {
      "student_id": {
        "type": "string",
        "pattern": "^[0-9]{10}$",
        "description": "Must equal the authenticated session's student ID."
      },
      "category": {
        "type": "string",
        "enum": ["missing_marks", "retake_request", "registration_issue", "timetable_clash"]
      },
      "subject": { "type": "string", "maxLength": 100 },
      "description": { "type": "string", "maxLength": 500 }
    },
    "required": ["student_id", "category", "subject", "description"],
    "additionalProperties": false
  }
}
```

Note the schema has no status property at all, and additionalProperties: false means a call that tries to smuggle in a status value is rejected as a schema violation before it reaches any handling code — this is the same “AI may draft, deterministic code decides the record’s real state” rule from the Week 1 AI Boundary Matrix, enforced at the schema level rather than only by instruction.

Output schema

```
{
  "type": "object",
  "properties": {
    "case_id": { "type": "string", "pattern": "^[CASE-[0-9]{4}$" },
    "status": { "type": "string", "const": "Pending" },
    "category": { "type": "string" },
    "subject": { "type": "string" },
    "created_by": { "type": "string", "const": "agent_draft" },
    "created_at": { "type": "string", "format": "date-time" }
  },
  "required": ["case_id", "status", "category", "subject", "created_by", "created_at"]
}
```

Authorization requirements: student_id must match the authenticated session (identical rule to the read tools above); category must be one of the four enumerated values — a request that implies a disciplinary, admissions, or fee-related category is intercepted by the orchestrator’s boundary check upstream of this tool and is redirected instead of being force-fit into one of the four categories.

Failure behavior: - Any attempt to pass a status, assigned_office, or other field outside the schema → rejected as invalid arguments; the tool is never invoked, and no partial record is created. - category outside the enum → validation error; agent falls back to a boundary/redirect response instead of guessing the closest category. - Missing subject or description → validation error naming the missing field; the agent asks the student for it rather than retrying with a placeholder. - Simulated side effect, explicitly scoped: a successful call only appends a JSON record to data/cases.json with status: "Pending". It sends no email, opens no ticket in a real system, and triggers no downstream workflow - it is “low-risk” precisely because its entire effect is fully reversible (the record can be deleted or ignored) and requires a separate, human-only, out-of-band action to become consequential.

3. Read vs. Write at a glance

	Retrieves current application data	Performs a side effect
get_timetable	(live timetable store)	-
get_case_status	(live case store)	-
search_knowledge_base	(static reference corpus)	-
create_support_case	-	(low-risk, simulated: draft-only, status locked to “pending”, no external system touched)

4. Deliberately Excluded Tools

The Week 3 grounding-failure report found that phrasing-based boundary detection can be bypassed (R13: “admission into” was not recognized as an admissions request; R14 asked for a direct grade change; R11 showed a mixed request can confuse a single boundary check). The more durable mitigation is architectural: the tool registry passed to the model simply does not contain any function capable of the prohibited action, so even a successful prompt-injection or boundary-detection miss cannot produce a real state change.

Tool that does not exist	Why it is excluded
approve_case / resolve_case / reject_case	Any status change beyond “Pending” is a human-only action per the Week 1 AI Boundary Matrix; no function-calling path to it exists.
update_grade	Grade changes are never an agent action, regardless of how the request is phrased (this is exactly what R14 asked for).

approve_admission	Admissions decisions are out of scope entirely (Project Charter §4); this is exactly what R13 asked for.
waive_fee / approve_fee_waiver	Fee decisions are Bursar-only per the corpus's boundary notice (DOC-10); no tool exposes this capability.
delete_case / modify_case_record	The AI Boundary Matrix reserves deletion and modification of existing case records for deterministic software under human control, not the agent.